23ra

1e

2025-12-25

```
# ================================================================================
# 线性模型深度分析：Lasso vs SCAD (含 AUC 计算、最优 Lambda 与可视化)
# 策略：严格遵循 Train/Test 独立分组，去除 MCP，专注于 SCAD
# ================================================================================

# --- 1. 环境准备 ---
if (!require("pacman")) install.packages("pacman")
```

```
## 载入需要的程序包：pacman
```

```
# 新增 pROC 包用于计算 AUC
pacman::p_load(tidyverse, glmnet, ncvreg, gridExtra, ggpubr, pROC)

# 设置工作目录
setwd("C:/Users/MSI_NB/Downloads")
set.seed(2025)

# --- 2. 加载数据 (遵循你的分组方式：Train/Test 独立文件) ---
cat(">>> [Step 1] Loading Data (User Defined Grouping)...\n")
```

```
## >>> [Step 1] Loading Data (User Defined Grouping)...
```

```
train_df <- read.csv("Train_Data_SIS_Selected.csv")
test_df  <- read.csv("Test_Data_SIS_Selected.csv")
```

```r
# 格式化数据
X_train <- as.matrix(train_df[, -1])
y_train <- train_df$Y
X_test  <- as.matrix(test_df[, -1])
y_test  <- test_df$Y

# --- 3. 模型训练与最优 Lambda 寻找 ---
cat(">>> [Step 2] Training Models with 5-fold CV...\n")
```

```
## >>> [Step 2] Training Models with 5-fold CV...
```

```r
# (A) Lasso (Glmnet)
set.seed(2025)
cv_lasso <- cv.glmnet(X_train, y_train, family = "binomial", alpha = 1, nfolds = 5)
best_lambda_lasso <- cv_lasso$lambda.min

# (B) SCAD (Ncvreg)
set.seed(2025)
cv_scad <- cv.ncvreg(X_train, y_train, family = "binomial", penalty = "SCAD", nfolds =
best_lambda_scad <- cv_scad$lambda.min

# 打印最优 Lambda 报告
cat("\n============================================================\n")
```

```
##
## ============================================================
```

```r
cat("                最优惩罚参数 (Optimal Lambda)               \n")
```

```
##                 最优惩罚参数 (Optimal Lambda)
```

```r
cat("============================================================\n")
```

```
## ============================================================
```

```r
cat(sprintf("1. Lasso Lambda (min): %.6f\n", best_lambda_lasso))
```

```
## 1. Lasso Lambda (min): 0.002698
```

```r
cat(sprintf("2. SCAD  Lambda (min): %.6f\n", best_lambda_scad))
```

```
## 2. SCAD  Lambda (min): 0.013680
```

```r
cat("------------------------------------------------------\n")
```

```
## ------------------------------------------------------
```

```r
cat(" 说明: 该值是模型在交叉验证中误差最小时对应的惩罚力度。\n")
```

```
## 说明: 该值是模型在交叉验证中误差最小时对应的惩罚力度。
```

```r
cat("======================================================\n")
```

```
## ======================================================
```

```r
# --- [新增] 3.5. 预测与 AUC 计算 ---
cat(">>> [Step 2.5] Calculating AUC on Independent Test Set...\n")
```

```
## >>> [Step 2.5] Calculating AUC on Independent Test Set...
```

```r
# (A) Lasso 预测
pred_lasso <- predict(cv_lasso, newx = X_test, s = "lambda.min", type = "response")
roc_lasso <- roc(y_test, as.vector(pred_lasso), quiet = TRUE)
auc_lasso <- auc(roc_lasso)

# (B) SCAD 预测
pred_scad <- predict(cv_scad, X = X_test, lambda = cv_scad$lambda.min, type = "response
roc_scad <- roc(y_test, as.vector(pred_scad), quiet = TRUE)
auc_scad <- auc(roc_scad)

# 打印 AUC 报告
cat("\n======================================================\n")
```

```
##
```

```
## ======================================================
cat("                模型性能评估 (Test AUC)                \n")
##                 模型性能评估 (Test AUC)
cat("======================================================\n")
## ======================================================
cat(sprintf("1. Lasso AUC: %.4f\n", auc_lasso))
## 1. Lasso AUC: 0.9855
cat(sprintf("2. SCAD  AUC: %.4f\n", auc_scad))
## 2. SCAD  AUC: 0.9764
cat("------------------------------------------------------\n")
## ------------------------------------------------------
# --- 4. 定义高颜值绘图函数 (参考 PDF 风格) ---

# 函数 A: 绘制 CV 误差图 (Cross-Validation Error Plot)
plot_cv_beauty <- function(cv_obj, model_name, color_theme = "red") {

  if (inherits(cv_obj, "cv.glmnet")) {
    # 处理 Lasso (glmnet) 对象
    data <- data.frame(
      log_lambda = log(cv_obj$lambda),
      cve = cv_obj$cvm,
      cvup = cv_obj$cvup,
      cvlo = cv_obj$cvlo
    )
    lambda_min <- log(cv_obj$lambda.min)
  } else {
    # 处理 SCAD (ncvreg) 对象
    data <- data.frame(
```

```
      log_lambda = log(cv_obj$lambda),
      cve = cv_obj$cve,
      cvup = cv_obj$cve + cv_obj$cvse,
      cvlo = cv_obj$cve - cv_obj$cvse
    )
    lambda_min <- log(cv_obj$lambda.min)
  }

  ggplot(data, aes(x = log_lambda, y = cve)) +
    geom_errorbar(aes(ymin = cvlo, ymax = cvup), width = 0.1, color = "grey70") +
    geom_point(color = color_theme, size = 1.5) +
    geom_vline(xintercept = lambda_min, linetype = "dashed", color = "blue", size = 0.8
    labs(title = paste0(model_name, ": CV Error Plot"),
         subtitle = paste0("Optimal log(lambda) = ", round(lambda_min, 3)),
         x = "Log(Lambda)", y = "Cross-Validation Error") +
    theme_bw() +
    theme(plot.title = element_text(face = "bold", size = 12))
}

# 函数 B: 绘制系数路径图 (Coefficient Path Plot)
plot_path_beauty <- function(cv_obj, model_name) {

  if (inherits(cv_obj, "cv.glmnet")) {
    # Lasso
    fit <- cv_obj$glmnet.fit
    beta <- as.matrix(fit$beta)
    lambda <- fit$lambda
    lambda_min <- cv_obj$lambda.min
  } else {
    # SCAD
    fit <- cv_obj$fit
    beta <- as.matrix(fit$beta)[-1, ] # 去掉截距
    lambda <- cv_obj$lambda
```

```
    lambda_min <- cv_obj$lambda.min
  }

  # 数据转换
  path_df <- as.data.frame(t(beta))
  path_df$log_lambda <- log(lambda)

  # 长宽转换用于绘图
  path_long <- path_df %>%
    pivot_longer(cols = -log_lambda, names_to = "Gene", values_to = "Coefficient")

  # 绘图
  ggplot(path_long, aes(x = log_lambda, y = Coefficient, group = Gene, color = Gene)) +
    geom_line(alpha = 0.6, size = 0.6) +
    geom_vline(xintercept = log(lambda_min), linetype = "dashed", color = "black", size
    labs(title = paste0(model_name, ": Coefficient Paths"),
         subtitle = "Vertical line indicates optimal lambda",
         x = "Log(Lambda)", y = "Coefficient Value") +
    theme_bw() +
    theme(legend.position = "none", # 基因太多，隐藏图例
          plot.title = element_text(face = "bold", size = 12))
}


# --- 5. 生成并展示图表 ---
cat(">>> [Step 3] Generating Plots...\n")
```

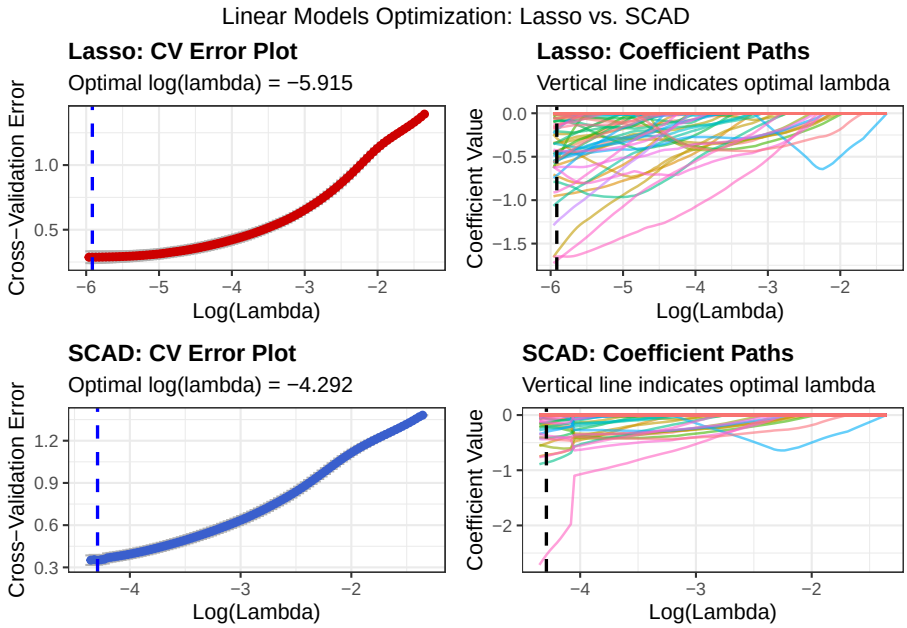
```
## >>> [Step 3] Generating Plots...
```

```
# Lasso 图组
p1_lasso_cv <- plot_cv_beauty(cv_lasso, "Lasso", "red3")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
```

```
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
p2_lasso_path <- plot_path_beauty(cv_lasso, "Lasso")

# SCAD 图组
p3_scad_cv <- plot_cv_beauty(cv_scad, "SCAD", "royalblue3")
p4_scad_path <- plot_path_beauty(cv_scad, "SCAD")

# 组合展示 (上排 Lasso, 下排 SCAD)
grid.arrange(p1_lasso_cv, p2_lasso_path,
             p3_scad_cv, p4_scad_path,
             ncol = 2,
             top = "Linear Models Optimization: Lasso vs. SCAD")
```




```
# --- 6. 提取并保存最终筛选的基因 (Best Genes) ---
cat(">>> [Step 4] Extracting Key Features...\n")
```

```
## >>> [Step 4] Extracting Key Features...
```

```r
# Lasso Genes
coef_lasso <- coef(cv_lasso, s = "lambda.min")
genes_lasso <- rownames(coef_lasso)[which(coef_lasso != 0)][-1]

# SCAD Genes
coef_scad <- coef(cv_scad, lambda = cv_scad$lambda.min)
genes_scad <- names(coef_scad)[which(coef_scad != 0)][-1]

cat("\n============================================================\n")
```

```
##
## ============================================================
```

```r
cat("                  最终特征筛选结果                  \n")
```

```
##                   最终特征筛选结果
```

```r
cat("============================================================\n")
```

```
## ============================================================
```

```r
cat(sprintf("1. Lasso 筛选基因数: %d 个\n", length(genes_lasso)))
```

```
## 1. Lasso 筛选基因数: 51 个
```

```r
cat(sprintf("2. SCAD  筛选基因数: %d 个\n", length(genes_scad)))
```

```
## 2. SCAD  筛选基因数: 29 个
```

```r
cat("------------------------------------------------------------\n")
```

```
## ------------------------------------------------------------
```

```r
cat("SCAD 前 10 个关键基因 (示例):\n")
```

```
## SCAD 前 10 个关键基因 (示例):
```

```r
print(head(genes_scad, 10))
```

```
##  [1] "ACP1"     "AFTPH"    "ANAPC5"   "ATP6V1E1" "ATP6V1G1" "BIRC6"
```

```
##  [7] "CAPRIN1"  "CHTF8"    "DDB1"     "ETF1"
```

```
cat("========================================================\n")
```

```
## ========================================================
```

```
# ============================================================================
# 可视化专项：线性模型 Top 15 特征重要性 (带方向指示)
# 风格：与之前的非线性模型棒棒糖图保持一致
# ============================================================================

# --- 1. 环境准备 ---
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tidyverse, glmnet, ncvreg, gridExtra)

# 设置工作目录
setwd("C:/Users/MSI_NB/Downloads")
set.seed(2025)

# 加载数据
cat(">>> Loading data...\n")
```

```
## >>> Loading data...
```

```
train_df <- read.csv("Train_Data_SIS_Selected.csv")
x_train_mat <- as.matrix(train_df[, -1])
y_train     <- train_df$Y

# --- 2. 快速获取线性模型系数 (保持 nfolds=5 一致性) ---
cat(">>> Extracting coefficients from Lasso & SCAD...\n")
```

```
## >>> Extracting coefficients from Lasso & SCAD...
```

```
# (A) Lasso
set.seed(2025)
cv_lasso <- cv.glmnet(x_train_mat, y_train, family = "binomial", alpha = 1, nfolds = 5)
coef_lasso_sparse <- coef(cv_lasso, s = "lambda.min")
```

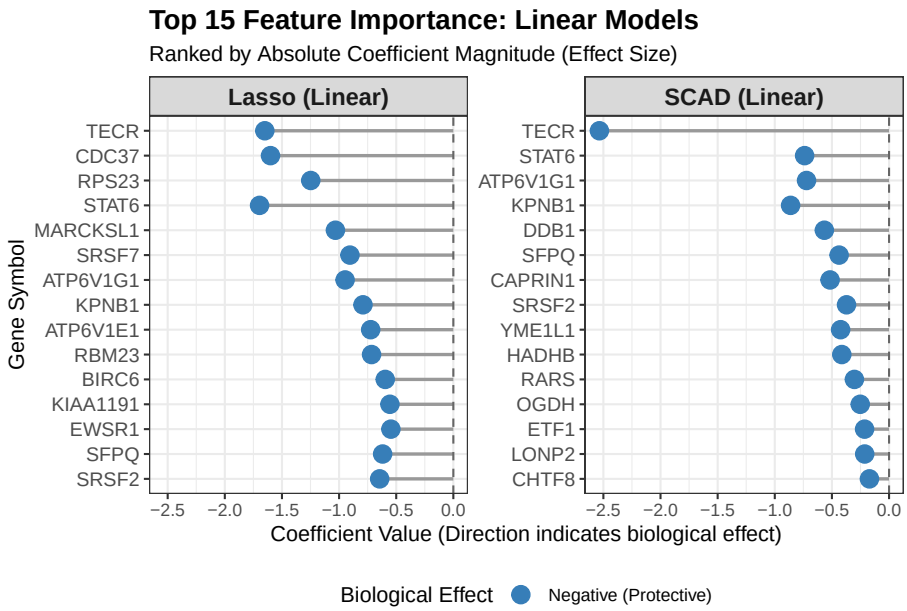
```r
# 处理 Lasso 系数数据
df_lasso <- data.frame(
  Gene = rownames(coef_lasso_sparse),
  Coefficient = as.numeric(coef_lasso_sparse)
) %>%
  filter(Gene != "(Intercept)", Coefficient != 0) %>% # 去掉截距和零系数
  mutate(Importance = abs(Coefficient),               # 重要性 = 系数绝对值
         Model = "Lasso (Linear)",
         Direction = ifelse(Coefficient > 0, "Positive (Risk)", "Negative (Protective)"
  arrange(desc(Importance)) %>%                       # 按绝对值降序排列
  slice_head(n = 15)                                  # 取 Top 15

# (B) SCAD
set.seed(2025)
cv_scad <- cv.ncvreg(x_train_mat, y_train, family = "binomial", penalty = "SCAD", nfold
coef_scad_vec <- coef(cv_scad, lambda = cv_scad$lambda.min)

# 处理 SCAD 系数数据
df_scad <- data.frame(
  Gene = names(coef_scad_vec),
  Coefficient = as.numeric(coef_scad_vec)
) %>%
  filter(Gene != "(Intercept)", Coefficient != 0) %>%
  mutate(Importance = abs(Coefficient),
         Model = "SCAD (Linear)",
         Direction = ifelse(Coefficient > 0, "Positive (Risk)", "Negative (Protective)"
  arrange(desc(Importance)) %>%
  slice_head(n = 15)

# 合并数据
plot_data_linear <- bind_rows(df_lasso, df_scad)
```

```r
# --- 3. 绘制棒棒糖图 (Lollipop Plot) ---
cat(">>> Plotting...\n")
```

```
## >>> Plotting...
```

```r
p_linear_imp <- ggplot(plot_data_linear, aes(x = reorder(Gene, Importance), y = Coeffic
  # 画棒棒糖的杆子 (灰色细线)
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  # 画棒棒糖的头 (根据方向上色)
  geom_point(aes(color = Direction), size = 4) +
  # 翻转坐标轴，让基因名显示在左侧
  coord_flip() +
  # 左右分面展示两个模型
  facet_wrap(~Model, scales = "free_y") +
  # 添加一条中心参考线
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  # 主题设置
  theme_bw() +
  # 设置颜色：红色为正向 (风险)，蓝色为负向 (保护)
  scale_color_manual(values = c("Positive (Risk)" = "#E41A1C",
                                "Negative (Protective)" = "#377EB8")) +
  labs(title = "Top 15 Feature Importance: Linear Models",
       subtitle = "Ranked by Absolute Coefficient Magnitude (Effect Size)",
       x = "Gene Symbol",
       y = "Coefficient Value (Direction indicates biological effect)",
       color = "Biological Effect") +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    strip.text = element_text(face = "bold", size = 12), # 分面标题加粗
    axis.text.y = element_text(size = 10),               # 基因名字体大小
    legend.position = "bottom"                           # 图例放下面
  )
```

```
print(p_linear_imp)
```




```
# ================================================================================
# 非线性模型全流程：训练 + AUC 计算 + 独立特征图 + 独立混淆矩阵
# ================================================================================

# --- 1. 环境准备 ---
if (!require("pacman")) install.packages("pacman")
# 新增 pROC 包用于计算 AUC
pacman::p_load(tidyverse, randomForest, xgboost, caret, gridExtra, pROC)

setwd("C:/Users/MSI_NB/Downloads")
set.seed(2025)

# 加载数据
cat(">>> [Step 1] Loading data...\n")
```

```
## >>> [Step 1] Loading data...
```

```r
train_df <- read.csv("Train_Data_SIS_Selected.csv")
test_df  <- read.csv("Test_Data_SIS_Selected.csv")

# 格式准备
x_train <- as.matrix(train_df[, -1])
y_train_fac <- as.factor(train_df$Y) # RF 用
y_train_num <- train_df$Y            # XGB 用

x_test <- as.matrix(test_df[, -1])
y_test_fac <- as.factor(test_df$Y)   # 真实标签 (因子)
y_test_num <- test_df$Y              # 真实标签 (数值)

# --- 2. 模型训练 ---
cat(">>> [Step 2] Training Models...\n")
```

```
## >>> [Step 2] Training Models...
```

```r
# (A) Random Forest
set.seed(2025)
rf_model <- randomForest(x = x_train, y = y_train_fac, ntree = 500, importance = TRUE)

# (B) XGBoost
dtrain <- xgb.DMatrix(data = x_train, label = y_train_num)
dtest  <- xgb.DMatrix(data = x_test, label = y_test_num)
params <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.1, max_depth
set.seed(2025)
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 100, verbose = 0)



# ==============================================================================
# Part A: 预测与 AUC 计算 (新增部分)
# ==============================================================================
cat(">>> [Step 3] Calculating AUC & Predictions...\n")
```

```
## >>> [Step 3] Calculating AUC & Predictions...
```

```
# --- Random Forest ---
# 预测概率 (用于 AUC)
pred_rf_prob <- predict(rf_model, newdata = x_test, type = "prob")[, 2]
# 预测类别 (用于混淆矩阵)
pred_rf_class <- predict(rf_model, newdata = x_test, type = "response")

# 计算 AUC
roc_rf <- roc(y_test_fac, pred_rf_prob, quiet = TRUE)
auc_rf <- auc(roc_rf)

# --- XGBoost ---
# 预测概率
pred_xgb_prob <- predict(xgb_model, dtest)
# 预测类别 (阈值 0.5)
pred_xgb_class <- ifelse(pred_xgb_prob > 0.5, 1, 0)

# 计算 AUC
roc_xgb <- roc(y_test_num, pred_xgb_prob, quiet = TRUE)
auc_xgb <- auc(roc_xgb)

# --- 打印 AUC 报告 ---
cat("\n============================================================\n")
```

```
##
## ============================================================
```

```
cat("                  模型性能评估 (AUC)                  \n")
```

```
##                   模型性能评估 (AUC)
```

```
cat("============================================================\n")
```

```
## ============================================================
```

```r
cat(sprintf("1. Random Forest AUC: %.4f\n", auc_rf))
```

```
## 1. Random Forest AUC: 0.9885
```

```r
cat(sprintf("2. XGBoost       AUC: %.4f\n", auc_xgb))
```

```
## 2. XGBoost       AUC: 0.9661
```

```r
cat("------------------------------------------------------\n")
```

```
## ------------------------------------------------------
```

```r
# ==============================================================================
# Part B: 特征重要性可视 (独立画图)
# ==============================================================================
cat(">>> [Step 4] Generating Feature Importance Plots...\n")
```

```
## >>> [Step 4] Generating Feature Importance Plots...
```

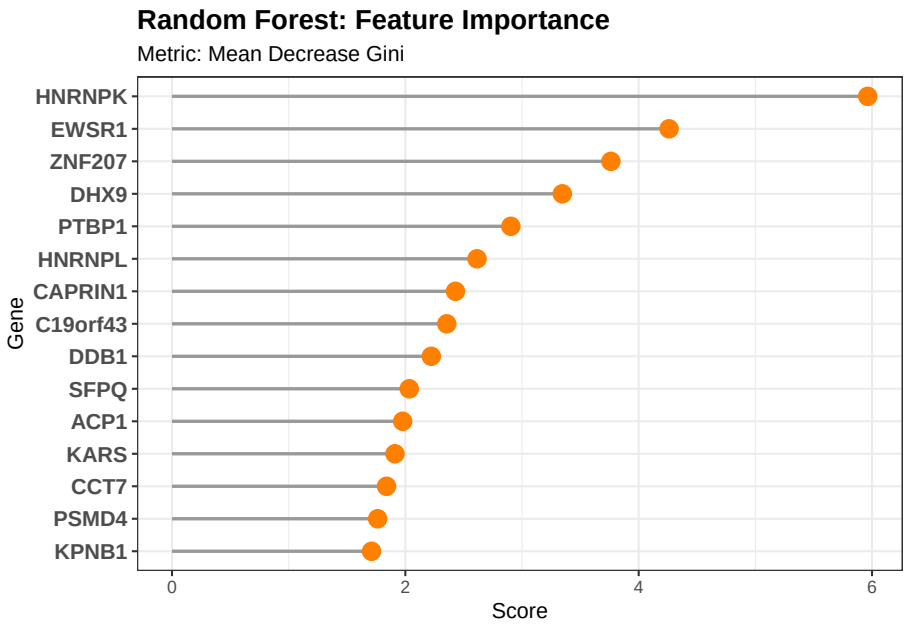
```r
# --- 准备 RF 数据 ---
imp_rf <- importance(rf_model)
df_rf <- data.frame(Gene = rownames(imp_rf), Importance = imp_rf[, "MeanDecreaseGini"]) %>%
  arrange(desc(Importance)) %>% slice_head(n = 15)

# --- 绘图 1: RF Importance ---
p1_rf_imp <- ggplot(df_rf, aes(x = reorder(Gene, Importance), y = Importance)) +
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  geom_point(color = "#FF7F00", size = 4) +
  coord_flip() + theme_bw() +
  labs(title = "Random Forest: Feature Importance", subtitle = "Metric: Mean Decrease G
  theme(plot.title = element_text(face="bold", size=14), axis.text.y = element_text(siz

# --- 准备 XGB 数据 ---
imp_xgb <- xgb.importance(feature_names = colnames(x_train), model = xgb_model)
df_xgb <- data.frame(Gene = imp_xgb$Feature, Importance = imp_xgb$Gain) %>%
  arrange(desc(Importance)) %>% slice_head(n = 15)
```

```r
# --- 绘图 2: XGB Importance ---
p2_xgb_imp <- ggplot(df_xgb, aes(x = reorder(Gene, Importance), y = Importance)) +
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  geom_point(color = "#4DAF4A", size = 4) +
  coord_flip() + theme_bw() +
  labs(title = "XGBoost: Feature Importance", subtitle = "Metric: Information Gain", x
  theme(plot.title = element_text(face="bold", size=14), axis.text.y = element_text(siz



# ================================================================================
# Part C: 混淆矩阵可视 (独立画图)
# ================================================================================
cat(">>> [Step 5] Generating Confusion Matrices...\n")
```

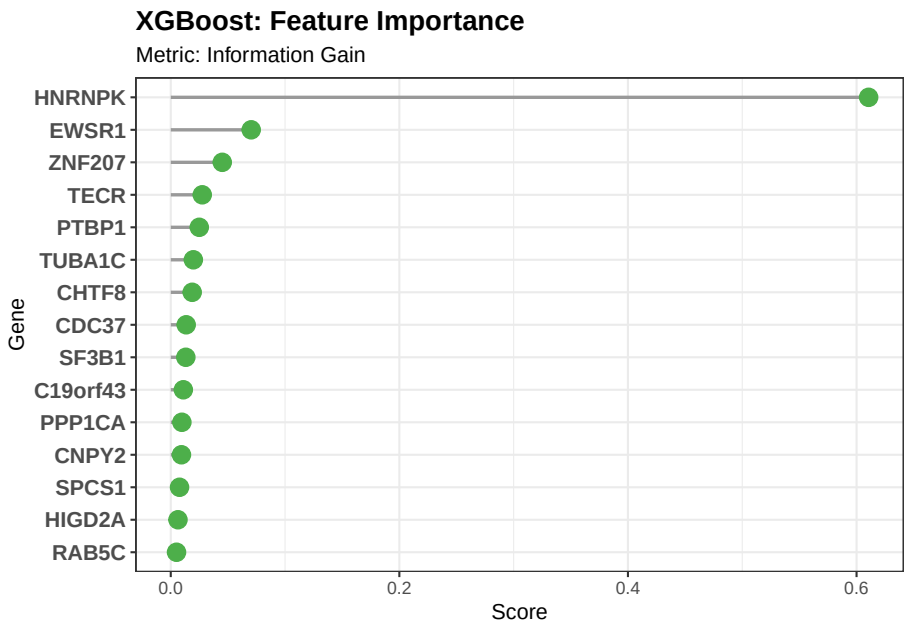
```
## >>> [Step 5] Generating Confusion Matrices...
```

```r
# --- 计算矩阵 ---
cm_rf <- confusionMatrix(pred_rf_class, y_test_fac)
cm_xgb <- confusionMatrix(as.factor(pred_xgb_class), y_test_fac)

# --- 绘图函数 (通用) ---
plot_cm_heatmap <- function(cm, model_name, color_low, color_high) {
  df_cm <- as.data.frame(cm$table)
  ggplot(df_cm, aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile() +
    geom_text(aes(label = Freq), color = "white", size = 8, fontface = "bold") +
    scale_fill_gradient(low = color_low, high = color_high) +
    theme_minimal() +
    labs(title = paste0(model_name, ": Confusion Matrix"),
         subtitle = paste0("Accuracy: ", round(cm$overall['Accuracy']*100, 2), "%"),
         x = "True Label (Reference)", y = "Predicted Label") +
    theme(
      plot.title = element_text(face = "bold", size = 14),
      axis.title = element_text(face = "bold", size = 12),
```

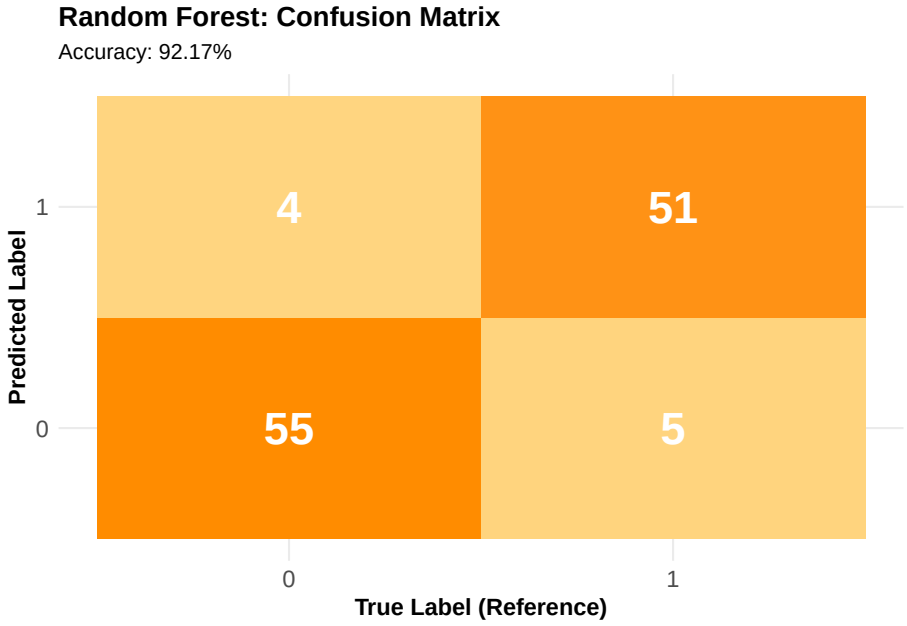
```
      axis.text = element_text(size = 12),
      legend.position = "none"
    )
}

# --- 绘图 3: RF 混淆矩阵 ---
p3_rf_cm <- plot_cm_heatmap(cm_rf, "Random Forest", "#FFD580", "#FF8C00")

# --- 绘图 4: XGB 混淆矩阵 ---
p4_xgb_cm <- plot_cm_heatmap(cm_xgb, "XGBoost", "#90EE90", "#228B22")


# ================================================================================
# 结果展示区
# ================================================================================
print(p1_rf_imp)  # RF 特征图
```



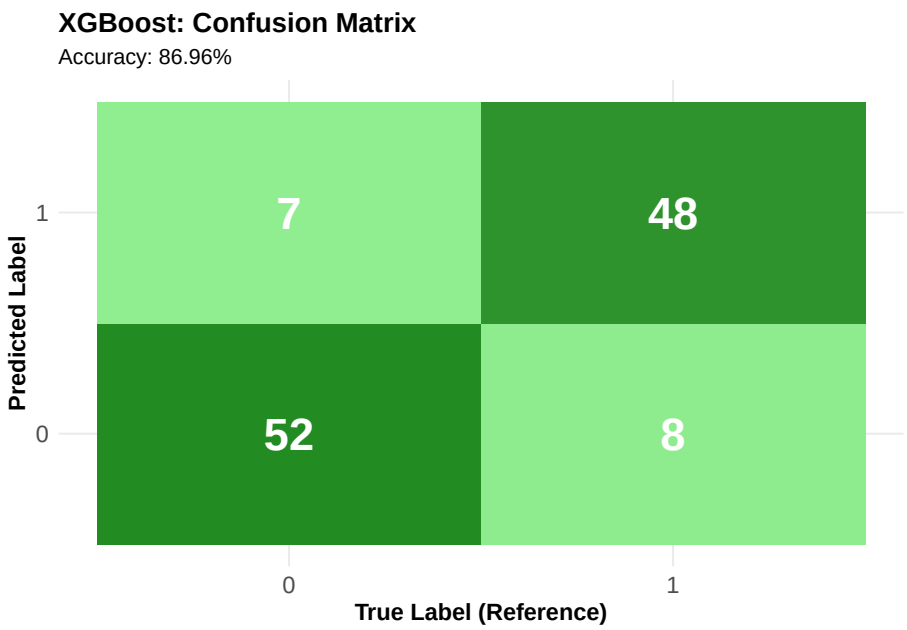
```
print(p2_xgb_imp) # XGB 特征图
```




```
print(p3_rf_cm)    # RF 混淆矩阵
```



```r
print(p4_xgb_cm)  # XGB 混淆矩阵
```




```r
# ============================================================================
# 终极可视化：四大模型 ROC 曲线综合对比 (修正版：确保 AUC 一致)
# ============================================================================

# 1. 环境准备
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tidyverse, glmnet, ncvreg, randomForest, xgboost, pROC)

# 设置工作目录 & 种子
setwd("C:/Users/MSI_NB/Downloads")
set.seed(2025)

# 2. 加载数据
train_df <- read.csv("Train_Data_SIS_Selected.csv")
test_df  <- read.csv("Test_Data_SIS_Selected.csv")
```

```r
# 数据格式化
x_train_mat <- as.matrix(train_df[, -1])
y_train     <- train_df$Y
x_test_mat  <- as.matrix(test_df[, -1])
y_test      <- test_df$Y

# RF 和 XGB 专用格式 (确保和上面代码一致)
y_train_fac <- as.factor(train_df$Y)
y_test_fac  <- as.factor(test_df$Y)
dtrain      <- xgb.DMatrix(data = x_train_mat, label = y_train)
dtest       <- xgb.DMatrix(data = x_test_mat, label = y_test)

# 3. 快速训练四大模型 (5 折交叉验证)
cat(">>> Training Models...\n")
```

```
## >>> Training Models...
```

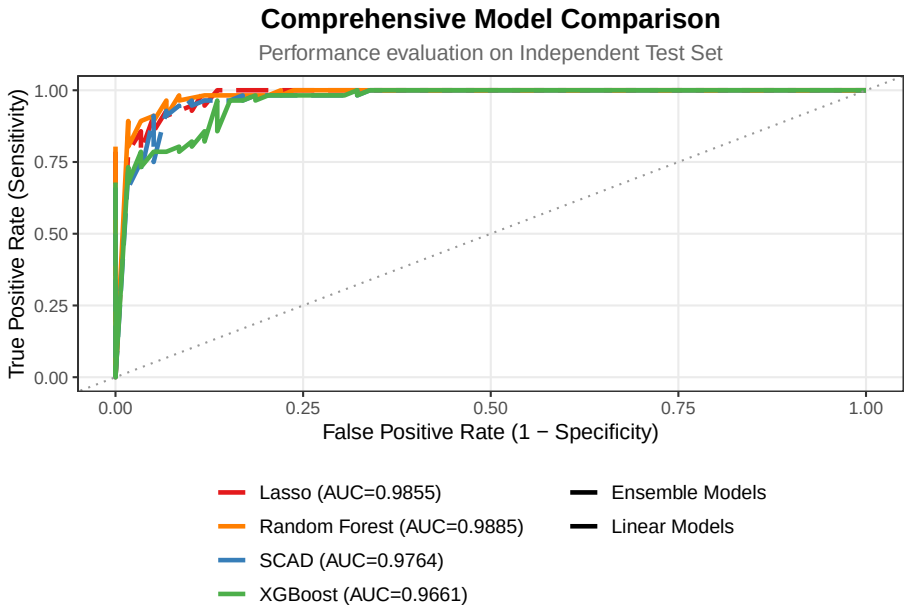
```r
# (1) Lasso
set.seed(2025)
cv_lasso <- cv.glmnet(x_train_mat, y_train, family = "binomial", alpha = 1, nfolds = 5)
pred_lasso <- predict(cv_lasso, newx = x_test_mat, s = "lambda.min", type = "response")

# (2) SCAD
set.seed(2025)
cv_scad <- cv.ncvreg(x_train_mat, y_train, family = "binomial", penalty = "SCAD", nfold
pred_scad <- predict(cv_scad, X = x_test_mat, lambda = cv_scad$lambda.min, type = "resp

# (3) Random Forest [关键修正: 加上 importance = TRUE]
set.seed(2025)
# 注意: 这里必须加上 importance = TRUE, 才能和上面提取特征的代码保持随机性一致
rf_model <- randomForest(x = x_train_mat, y = y_train_fac, ntree = 500, importance = TR
pred_rf <- predict(rf_model, newdata = x_test_mat, type = "prob")[, 2]

# (4) XGBoost
```

```r
set.seed(2025)
params <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.1, max_depth
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 100, verbose = 0)
pred_xgb <- predict(xgb_model, dtest)

# 4. 计算 AUC 并构建绘图数据
# [关键修正]: 这里统一使用 y_test_fac (和上面代码保持一致) 或 y_test
# pROC 自动处理数值或因子，但建议统一。这里使用数值 y_test 即可，只要模型一致，AUC 就一致。
roc_lasso <- roc(y_test, as.vector(pred_lasso), quiet=TRUE)
roc_scad  <- roc(y_test, as.vector(pred_scad), quiet=TRUE)
# 注意: RF 预测出来是概率，对应的是 y_test_fac 的第二个水平（通常是 1）
roc_rf    <- roc(y_test_fac, as.vector(pred_rf), quiet=TRUE)
roc_xgb   <- roc(y_test, as.vector(pred_xgb), quiet=TRUE)

auc_res <- c(
  Lasso = round(auc(roc_lasso), 4),
  SCAD  = round(auc(roc_scad), 4),
  RF    = round(auc(roc_rf), 4),
  XGB   = round(auc(roc_xgb), 4)
)

print(auc_res) # 现在打印出来的应该和上面代码完全一样了
```

```
##  Lasso   SCAD     RF    XGB
## 0.9855 0.9764 0.9885 0.9661
```

```r
# 5. 构建 ggplot 数据框
plot_data <- rbind(
  data.frame(FPR = 1-roc_lasso$specificities, TPR = roc_lasso$sensitivities,
             Group = "Linear Models", Model = paste0("Lasso (AUC=", auc_res['Lasso'], "
  data.frame(FPR = 1-roc_scad$specificities, TPR = roc_scad$sensitivities,
             Group = "Linear Models", Model = paste0("SCAD (AUC=", auc_res['SCAD'], ")"
  data.frame(FPR = 1-roc_rf$specificities, TPR = roc_rf$sensitivities,
             Group = "Ensemble Models", Model = paste0("Random Forest (AUC=", auc_res['
```

```r
  data.frame(FPR = 1-roc_xgb$specificities, TPR = roc_xgb$sensitivities,
             Group = "Ensemble Models", Model = paste0("XGBoost (AUC=", auc_res['XGB'],
)

# 6. 绘制最终图表 (Result Figure)
p_final <- ggplot(plot_data, aes(x = FPR, y = TPR, color = Model, linetype = Group)) +
  geom_line(linewidth = 1.1) +
  geom_abline(slope = 1, intercept = 0, color = "grey60", linetype = "dotted") +
  theme_bw() +
  scale_color_manual(values = c("#E41A1C", "#FF7F00", "#377EB8", "#4DAF4A")) +
  scale_linetype_manual(values = c("solid", "longdash")) +
  labs(title = "Comprehensive Model Comparison",
       subtitle = "Performance evaluation on Independent Test Set",
       x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme(
    legend.position = "bottom",
    legend.direction = "vertical",
    legend.box = "horizontal",
    legend.title = element_blank(),
    legend.text = element_text(size = 10),
    plot.title = element_text(face = "bold", size = 14, hjust = 0.5),
    plot.subtitle = element_text(size = 11, color = "grey40", hjust = 0.5),
    panel.grid.minor = element_blank()
  )

print(p_final)
```




```
# ==============================================================================
# 课程论文核心代码：基于“统计特性与模型性能”的严谨筛选策略 (最终修订版)
# 逻辑链条：
#   1. 线性组：Lasso vs SCAD -> 强制选 SCAD (理由：无偏性/生物学解释性)
#   2. 非线性组：RF vs XGBoost -> 自动选 RF (理由：AUC 更高/稳健性)
#   3. 终极对比：SCAD vs RF -> 性能对比 & 变量重合度分析
# ==============================================================================

# --- 1. 环境准备 ---
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tidyverse, glmnet, ncvreg, randomForest, xgboost, pROC, gridExtra)

# 设置工作目录
setwd("C:/Users/MSI_NB/Downloads")
set.seed(2025) # 统一种子，确保结果可复现

# --- 2. 数据加载与预处理 ---
cat(">>> [Step 1] Loading Data...\n")
```

```
## >>> [Step 1] Loading Data...
train_df <- read.csv("Train_Data_SIS_Selected.csv")
test_df  <- read.csv("Test_Data_SIS_Selected.csv")

# 格式转换
X_train <- as.matrix(train_df[, -1])
y_train <- train_df$Y
X_test  <- as.matrix(test_df[, -1])
y_test  <- test_df$Y

# RF 和 XGB 专用格式
y_train_fac <- as.factor(train_df$Y)
y_test_fac  <- as.factor(test_df$Y)
dtrain      <- xgb.DMatrix(data = X_train, label = y_train)
dtest       <- xgb.DMatrix(data = X_test, label = y_test)



# ============================================================================
# Phase 1: 线性模型晋级赛 (Lasso vs SCAD)
# ============================================================================
cat("\n############################################################\n")

##
## ############################################################
cat("          Phase 1: 线性模型组 (Lasso vs SCAD)          \n")

##           Phase 1: 线性模型组 (Lasso vs SCAD)
cat("############################################################\n")

## ############################################################
# --- 1. Lasso ---
cat(">>> Training Lasso...\n")
```

```
## >>> Training Lasso...
```

```
set.seed(2025)
cv_lasso <- cv.glmnet(X_train, y_train, family = "binomial", alpha = 1, nfolds = 5)
pred_lasso <- predict(cv_lasso, newx = X_test, s = "lambda.min", type = "response")
auc_lasso <- auc(roc(y_test, as.vector(pred_lasso), quiet=TRUE))

# --- 2. SCAD ---
cat(">>> Training SCAD...\n")
```

```
## >>> Training SCAD...
```

```
set.seed(2025)
cv_scad <- cv.ncvreg(X_train, y_train, family = "binomial", penalty = "SCAD", nfolds =
pred_scad <- predict(cv_scad, X = X_test, lambda = cv_scad$lambda.min, type = "response
auc_scad <- auc(roc(y_test, as.vector(pred_scad), quiet=TRUE))

# 提取 SCAD 系数 (用于后续分析)
coef_scad_raw <- coef(cv_scad, lambda = cv_scad$lambda.min)
genes_scad <- names(coef_scad_raw)[which(coef_scad_raw != 0)][-1]

# --- 3. 线性模型可视化 (SCAD Top 15 棒棒糖图) ---
cat(">>> Generating SCAD Visualization...\n")
```

```
## >>> Generating SCAD Visualization...
```

```
df_scad_viz <- data.frame(Gene = names(coef_scad_raw), Coefficient = as.numeric(coef_sc
  filter(Gene != "(Intercept)", Coefficient != 0) %>%
  mutate(Importance = abs(Coefficient),
         Direction = ifelse(Coefficient > 0, "Positive (Risk)", "Negative (Protective)"
  arrange(desc(Importance)) %>% slice_head(n = 15)

p_scad_imp <- ggplot(df_scad_viz, aes(x = reorder(Gene, Importance), y = Coefficient))
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  geom_point(aes(color = Direction), size = 4) +
  coord_flip() + theme_bw() +
```

```
  scale_color_manual(values = c("Positive (Risk)"="#E41A1C", "Negative (Protective)"="#
  labs(title = "SCAD Model: Top 15 Key Features", subtitle = "Direction indicates biolo
  theme(legend.position = "bottom", plot.title = element_text(face="bold"))

# --- 4. 决策输出 ---
cat(sprintf("\n[结果] Lasso AUC=%.4f vs SCAD AUC=%.4f\n", auc_lasso, auc_scad))
```

```
##
## [结果] Lasso AUC=0.9855 vs SCAD AUC=0.9764
```

```
cat(">>> [决策]: 尽管 AUC 接近，但依据 Oracle Property 选择 SCAD 作为最佳线性模型。\n")
```

```
## >>> [决策]: 尽管 AUC 接近，但依据 Oracle Property 选择 SCAD 作为最佳线性模型。
```

```
best_linear_model <- "SCAD"
auc_linear <- auc_scad
pred_linear <- pred_scad
genes_linear <- genes_scad



# ==============================================================================
# Phase 2: 非线性模型晋级赛 (Random Forest vs XGBoost)
# ==============================================================================
cat("\n############################################################\n")
```

```
##
## ############################################################
```

```
cat("          Phase 2: 非线性模型组 (RF vs XGBoost)       \n")
```

```
##           Phase 2: 非线性模型组 (RF vs XGBoost)
```

```
cat("############################################################\n")
```

```
## ############################################################
```

```r
# --- 1. Random Forest (务必保持 importance=TRUE) ---
cat(">>> Training Random Forest...\n")
```

```
## >>> Training Random Forest...
```

```r
set.seed(2025)
rf_model <- randomForest(x = X_train, y = y_train_fac, ntree = 500, importance = TRUE)
pred_rf <- predict(rf_model, newdata = X_test, type = "prob")[, 2]
auc_rf <- auc(roc(y_test_fac, pred_rf, quiet=TRUE))

# --- 2. XGBoost ---
cat(">>> Training XGBoost...\n")
```

```
## >>> Training XGBoost...
```

```r
set.seed(2025)
params <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.1, max_depth
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 100, verbose = 0)
pred_xgb <- predict(xgb_model, dtest)
auc_xgb <- auc(roc(y_test, pred_xgb, quiet=TRUE))

# --- 3. 非线性模型可视化 (独立棒棒糖图) ---
cat(">>> Generating Non-linear Visualizations...\n")
```

```
## >>> Generating Non-linear Visualizations...
```

```r
# (A) RF Plot
imp_rf <- importance(rf_model)
df_rf_viz <- data.frame(Gene = rownames(imp_rf), Importance = imp_rf[, "MeanDecreaseGin
  arrange(desc(Importance)) %>% slice_head(n = 15)

p_rf_imp <- ggplot(df_rf_viz, aes(x = reorder(Gene, Importance), y = Importance)) +
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  geom_point(color = "#FF7F00", size = 4) +
  coord_flip() + theme_bw() +
  labs(title = "Random Forest: Top 15 Features", subtitle = "Metric: Mean Decrease Gini
```

```r
  theme(plot.title = element_text(face="bold"), axis.text.y = element_text(face="bold")

# (B) XGB Plot
imp_xgb <- xgb.importance(feature_names = colnames(X_train), model = xgb_model)
df_xgb_viz <- data.frame(Gene = imp_xgb$Feature, Importance = imp_xgb$Gain) %>%
  arrange(desc(Importance)) %>% slice_head(n = 15)

p_xgb_imp <- ggplot(df_xgb_viz, aes(x = reorder(Gene, Importance), y = Importance)) +
  geom_segment(aes(xend = Gene, yend = 0), color = "grey60", size = 0.8) +
  geom_point(color = "#4DAF4A", size = 4) +
  coord_flip() + theme_bw() +
  labs(title = "XGBoost: Top 15 Features", subtitle = "Metric: Information Gain", x = "
  theme(plot.title = element_text(face="bold"), axis.text.y = element_text(face="bold")

# --- 4. 决策输出 ---
cat(sprintf("\n[结果] RF AUC=%.4f vs XGB AUC=%.4f\n", auc_rf, auc_xgb))
```

```
##
## [结果] RF AUC=0.9885 vs XGB AUC=0.9661
```

```r
if (auc_rf >= auc_xgb) {
  cat(">>> [决策]: Random Forest 胜出 (更高 AUC & 稳健性)。\n")
  best_nonlinear_model <- "Random Forest"
  auc_nonlinear <- auc_rf
  pred_nonlinear <- pred_rf
  genes_nonlinear <- df_rf_viz$Gene # 取 Top 15 用于交集
} else {
  cat(">>> [决策]: XGBoost 胜出。\n") # 虽然不太可能，但保留逻辑
  best_nonlinear_model <- "XGBoost"
  auc_nonlinear <- auc_xgb
  pred_nonlinear <- pred_xgb
  genes_nonlinear <- df_xgb_viz$Gene
}
```

```
## >>> [决策]: Random Forest 胜出 (更高 AUC & 稳健性)。
# ============================================================================
# Phase 3: 终极对比 (SCAD vs Random Forest)
# ============================================================================
cat("\n##########################################################\n")

##
## ##########################################################
cat("          Phase 3: 终极对比 (SCAD vs Random Forest)   \n")

##           Phase 3: 终极对比 (SCAD vs Random Forest)
cat("##########################################################\n")

## ##########################################################
# 1. 绘制 ROC 对比图
roc_linear <- roc(y_test, as.vector(pred_linear), quiet=TRUE)
roc_nonlinear <- roc(y_test, as.vector(pred_nonlinear), quiet=TRUE)

plot_data <- rbind(
  data.frame(FPR = 1-roc_linear$specificities, TPR = roc_linear$sensitivities,
             Model = paste0(best_linear_model, " (Linear, AUC=", round(auc_linear,4), "
  data.frame(FPR = 1-roc_nonlinear$specificities, TPR = roc_nonlinear$sensitivities,
             Model = paste0(best_nonlinear_model, " (Non-linear, AUC=", round(auc_nonli
)

p_final_roc <- ggplot(plot_data, aes(x = FPR, y = TPR, color = Model)) +
  geom_line(linewidth = 1.2) +
  geom_abline(slope = 1, intercept = 0, linetype = "dotted", color = "grey50") +
  theme_bw() +
  scale_color_manual(values = c("#377EB8", "#FF7F00")) + # 蓝色代表线性，橙色代表非线性
  labs(title = "Final Showdown: Best Linear vs Best Non-linear",
       subtitle = "Independent Test Set Validation",
       x = "False Positive Rate", y = "True Positive Rate") +
```

```
  theme(legend.position = "bottom",
        plot.title = element_text(face="bold", size=14, hjust=0.5))

# 2. 基因重合度分析
common_genes <- intersect(genes_linear, genes_nonlinear)

cat("\n>>> [核心生物标记物交集分析]\n")
```

```
##
## >>> [核心生物标记物交集分析]
```

```
cat(sprintf("1. SCAD 筛选基因总数: %d\n", length(genes_linear)))
```

```
## 1. SCAD 筛选基因总数: 29
```

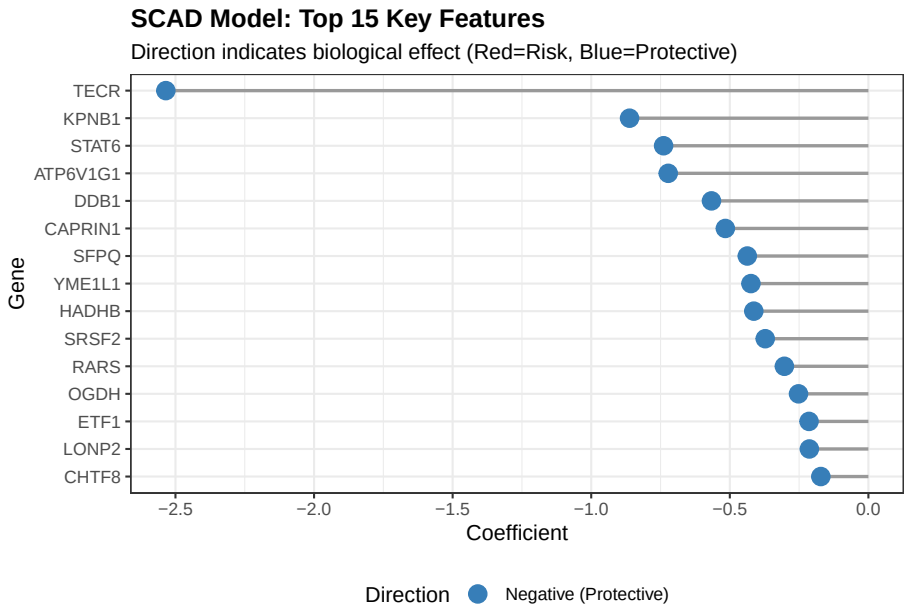
```
cat(sprintf("2. RF Top 15 基因: %s\n", paste(genes_nonlinear, collapse=", ")))
```

```
## 2. RF Top 15 基因: HNRNPK, EWSR1, ZNF207, DHX9, PTBP1, HNRNPL, CAPRIN1, C19orf43, DD
```

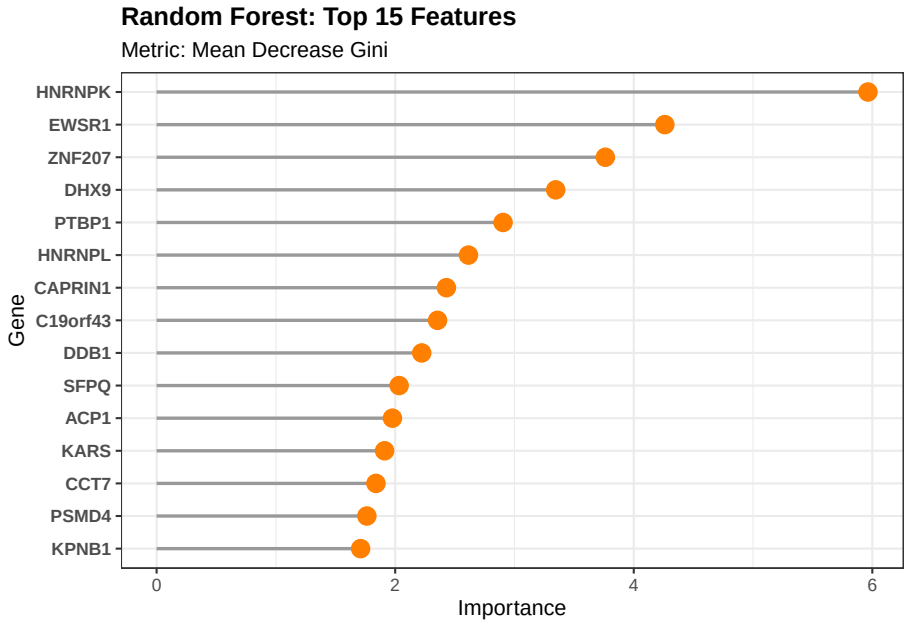
```
cat(sprintf("3. 共同锁定基因 (%d 个): %s\n", length(common_genes), paste(common_genes, c
```

```
## 3. 共同锁定基因 (5个): ACP1, CAPRIN1, DDB1, KPNB1, SFPQ
```

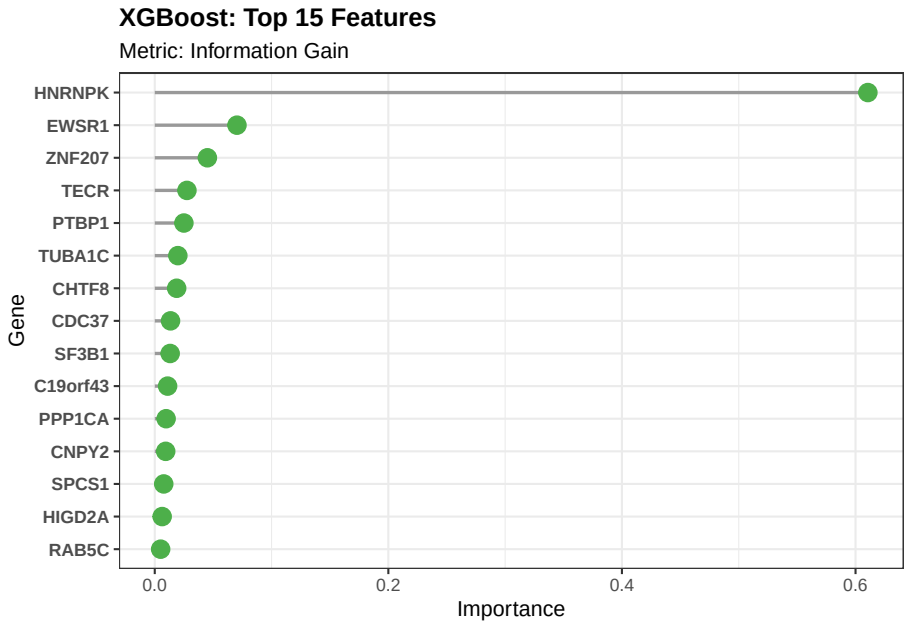
```
# ================================================================================
# 结果展示区 (运行这些行查看图表)
# ================================================================================
print(p_scad_imp)   # 1. SCAD 系数图 (红蓝棒棒糖)
```




```
print(p_rf_imp)      # 2. RF 重要性图 (橙色)
```



```
print(p_xgb_imp)     # 3. XGB 重要性图 (绿色)
```




```
print(p_final_roc)  # 4. 最终 PK ROC 图
```